The Vertex Subforest Sequence

Agastya Arora, Cora Shaklee, Rendi Adinata

July 2026

Abstract

We use an AI architecture called PatternBoost to produce counterexamples to the log-concavity condition in the vertex subforest sequence. The i-th element of the sequence is the number of vertex subsets of size i whose induced subgraph is a forest. We look only at bipartite graphs to reduce our search space, we encode them as one-dimensional objects by flattening the top-right block of the adjacency matrix. We train a transformer to generate high-scoring objects, where our scoring function is the log-concavity inequality on one side. We locally optimize the objects we get back from the transformer, and repeat. Our computation found 12 counterexamples. Our findings show how far combinatorial log-concavity extends beyond matroid-like settings. This has implications for knowing stronger conditions under which log-concavity holds.

1 Introduction

For a graph G, the sequence of subforest basis numbers $f_i(G)$ is the list of the number of valid forests found when all possible subgraphs of edges of size i are chosen. It has been shown that this sequence is always log-concave and satisfies the condition $a_i^2 \geq a_{i-1}a_{i+1}$; however, areas of discovery are presented when changing the sequence to count the number of vertex sets whose subgraph is a forest. The purpose of the research project recorded in this paper is to examine the sequence of vertex subforest numbers for bipartite graphs and to determine if examples that break log-concavity can be identified. This project used the machine learning architecture PatternBoost to explore counterexamples to log-concavity.

2 Our Code

2.1 Graph Representation

To limit storage requirements and reduce computing time, we chose to represent the bipartite graphs as lists. Given that a bipartite graph consists of two collections of vertices one with n nodes and the other with m nodes, the matrix representation consists of a $n \times m$ sub-matrix in the upper right corner and an $m \times n$ matrix in the lower left corner. The two matrices are transposes of each other and provide unique representations of bipartite graphs. Using this information, we flattened the upper sub-matrix into a list and appended the number

of columns onto the end of the list. This representation could easily be converted back to adjacency matrix form.

2.2 Creating and Testing Example Graphs

Our first step to test examples was to generate random matrices that represented valid bipartite graphs. For this file, we used the random library in Python to produce a random list of ones and zeros for a given vertex size.
Using the vertex size of the bipartite graph, the program computes all possible combinations for the chosen vertex subset size. This information is then used to create the sequence for the graph. A subsequent file checks if each subgraph for a number of vertices is a valid forest. Given that a sequence is log-concave if it satisfies $a_i^2 \geq a_{i-1}a_{i+1}$, the chosen scoring function computes $a_{i-1}a_{i+1} - a_i^2$ for every valid index in the sequence and returns the highest score. If the score exceeds zero, the graph has broken log-concavity.

2.3 PatternBoost

By compiling the individual Python files into one file, we created a PatternBoost environment. In this project, we used PatternBoost 2 as provided a more seamless process of creating an environment from our Python code.
Since PatternBoost requires the scoring function to return a positive value, we modified this code to use an exponential function. The new scoring function uses $e^{a_{i-1}a_{i+1}-a_i^2}$ where a score greater than one signifies a break in log-concavity.
Next, we needed a local optimization function to create examples closer to our desired outcome. Due to time constraints, we created a simple function that accepts the list representation of a graph and iterates through the values in the list. If the value is zero, then it is temporarily changed to a one and the scoring function is used. The index with the highest score is permanently changed to a one and this local search process is repeated a number of times.
After a few test runs of PatternBoost, we decided to implement a source of randomization to the local optimization function in order to prevent the model from generating the same graphs repeatedly. In this case, the function iterated through the local search process a random number of times. In addition, the code converted the value of a random index to zero. These changes allowed us to produce examples that break log-concavity.

3 Generated Counterexamples

Once we created the tokenizer and PatternBoost environment, we ran tests for different numbers of vertices. We originally tested graphs with 8, 9 and 10 nodes to ensure that the code was working correctly. Following this, we ran a test on 15 vertices as we expected graphs with higher vertices would be closer to breaking log concavity. This provided our first discovery of counterexamples.

3.1 15 Vertices

Since the adjacency matrix of a bipartite graph is represented as follows

$$A = \begin{bmatrix} 0 & B \\ B^T & 0 \end{bmatrix}$$

the small bipartite block B will represent the graphs for the subsequent examples. All of the 15 vertices counterexamples found in this research project have a bipartition of 7×8 and a break index of 12.

Counterexample 1.

$$B_1 = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

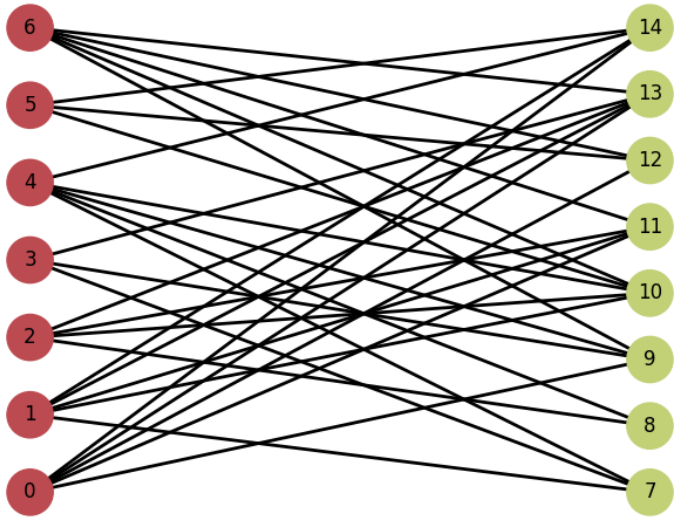


Figure 1: Bipartite counterexample generated in epoch 4 with a violation amount of 25.

Counterexample 2.

$$B_2 = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

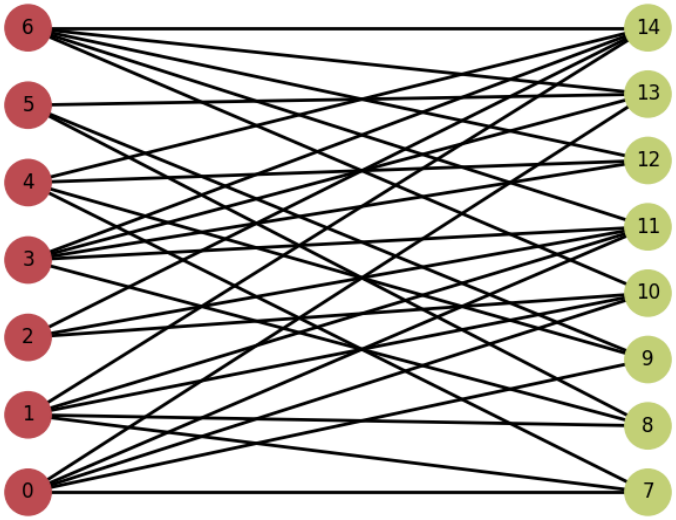


Figure 2: Bipartite counterexample generated in epoch 5 with a violation amount of 57.

Counterexample 3.

$$B_3 = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

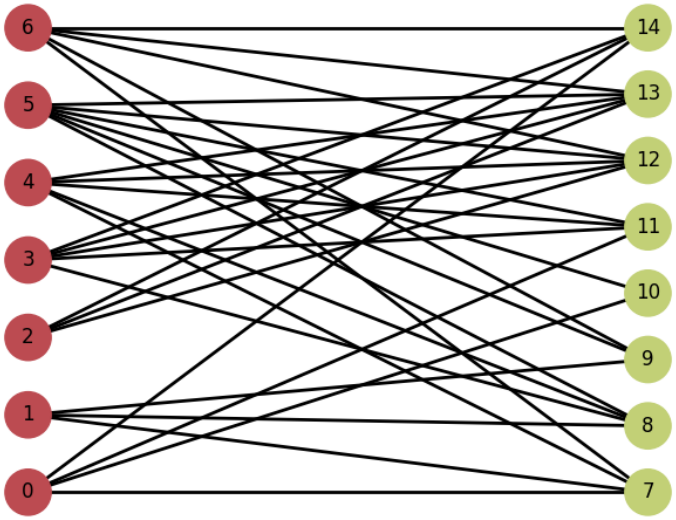


Figure 3: Bipartite counterexample generated in epoch 6 with a violation amount of 16.

Counterexample 4.

$$B_4 = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

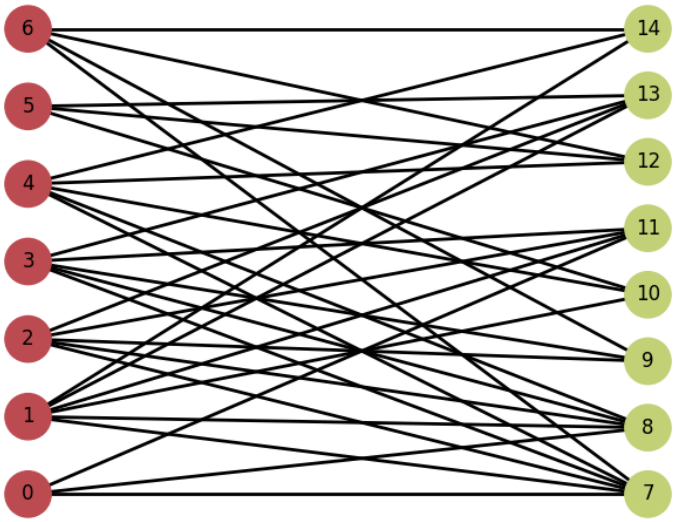


Figure 4: Bipartite counterexample generated in epoch 9 with a violation amount of 22.

Counterexample 5.

$$B_5 = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

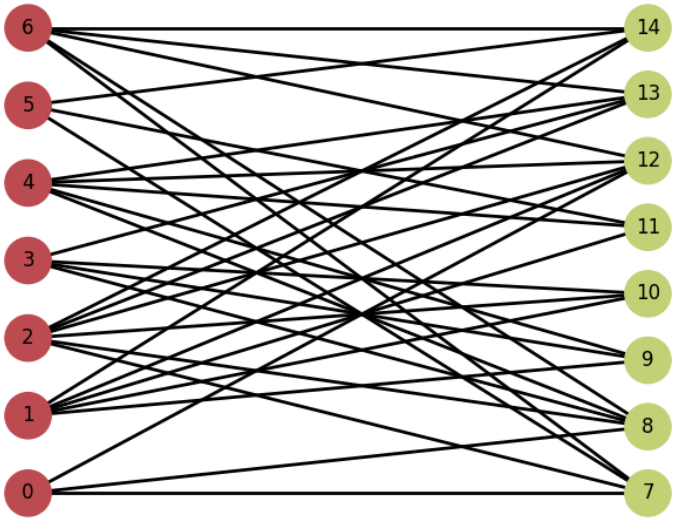


Figure 5: Bipartite counterexample generated in epoch 4 with a violation amount of 11.

Counterexample 6.

$$B_6 = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

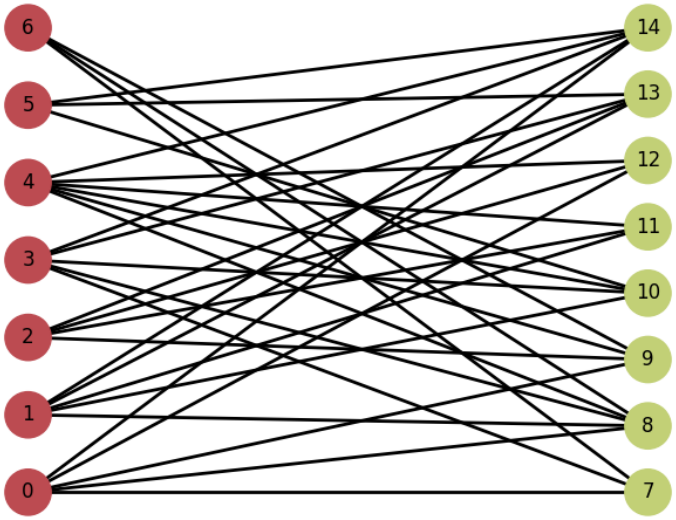


Figure 6: Bipartite counterexample generated in epoch 9 with a violation amount of 14.

4 Conclusion

For a bipartite graph G, we can generate a sequence of vertex subforest numbers. Using the PatternBoost 2 machine learning architecture, we successfully identified 12 bipartite graphs whose sequences break the log-concavity condition of $a_i^2 \geq a_{i-1}a_{i+1}$. Six unique graphs with 15 vertices and another set of six graphs with 14 vertices were tested and shown to be counterexamples.

Continued work on this research project could include testing higher vertices counts. Additionally, a more advanced local search function could provide faster and better generations of counterexamples. Simplifying the code in the local search and scoring functions as well as using a supercomputer would also make the tests faster. Further research into patterns associated with counterexamples could also provide valuable information.

5 GitHub Repositories

PatternBoost 2: https://github.com/AxiomMath/axplorer

TeamForest Repository: https://github.com/agastya-arora/TeamForest

TeamForest Axplorer Repository: https://github.com/agastya-arora/TeamForestAxplorer.git